

Algorithmes fondamentaux du projet

Identification d'un réseau de drogue

DIMBIN'NY AINA Nomenjanahary Andrianina Geoffroy

Mai 2026

Table des matières

1	Introduction	2
2	Algorithme 1 : Nettoyage du graphe	2
2.1	Objectif	2
2.2	Implémentation Python	2
2.3	Fonction trouver_composantes(G)	3
2.4	Fonction nettoyage(aretes, N)	3
2.5	Propriétés du graphe nettoyé	3
3	Algorithme 2 : Classification du graphe	3
3.1	Implémentation Python	3
3.2	Coloration gloutonne	5
3.2.1	Fonction colorier_graphe(G)	5
3.3	Identification des sous-graphes	5
3.3.1	Fonction identifier_sous_graphes(G)	5
3.4	Détection des cliques par Bron-Kerbosch	6
3.4.1	Fonction detecter_cliques(G, taille_min)	6
3.5	Interprétation criminologique des deux algorithmes	6

1 Introduction

Deux algorithmes fondamentaux structurent ce projet. Le premier assure le nettoyage du graphe brut. Le second réalise la classification des individus par coloration et détection de cliques.

2 Algorithme 1 : Nettoyage du graphe

2.1 Objectif

Transformer un graphe brut potentiellement sale (boucles, multi-arêtes, non connexe) en un graphe propre exploitable pour l'analyse criminologique.

2.2 Implémentation Python

```
1 import networkx as nx
2 from graphe import generate_random_graph
3
4 #Trouver les composantes connexes par un parcours en largeur
5 def trouver_composantes(G):
6     visited = set()
7     composantes = []
8
9     for depart in G.nodes():
10         if depart in visited:
11             continue
12
13         composante = set()
14         file_FIFO = [depart]
15
16         while file_FIFO:
17             sommet = file_FIFO.pop(0)
18             if sommet in visited:
19                 continue
20
21             visited.add(sommet)
22             composante.add(sommet)
23
24             for voisin in G.neighbors(sommet):
25                 if voisin not in visited:
26                     file_FIFO.append(voisin)
27
28             composantes.append(composante)
29
30     return composantes
31
32 #Nettoyage du graphe
33 def nettoyage(aretes, N):
34     G = nx.Graph()
35     G.add_nodes_from(range(N))
36
37     for u, v in aretes:
38         if u != v:
39             G.add_edge(u, v)
40
41     composantes = trouver_composantes(G)
42
43     if len(composantes) > 1:
44         noeud_ref = list(composantes[0])[0]
45         for composante in composantes[1:]:
46             noeud_composante = list(composante)[0]
47             G.add_edge(noeud_ref, noeud_composante)
48
49     return G
```

Listing 1 – algo1_nettoyage.py

2.3 Fonction trouver_composantes(G)

Cette fonction identifie toutes les composantes connexes du graphe.

[H] Détection des composantes connexes **Input** : G : un graphe avec un ensemble de sommets et d'arêtes

Output : Une liste d'ensembles contenant les sommets de chaque composante

$visites \leftarrow \emptyset$ $composantes \leftarrow []$

pour chaque depart dans $G.nodes()$ **faire**

si depart $\in$ visites **alors**

continuer

$composante \leftarrow \emptyset$ $file \leftarrow [depart]$

tant que file $\neq []$ **faire**

$sommet \leftarrow file.pop(0)$

si sommet $\in$ visites **alors**

continuer

$visites.add(sommet)$ $composante.add(sommet)$

pour chaque voisin dans $G.neighbors(sommet)$ **faire**

si voisin $\notin$ visites **alors**

$file.append(voisin)$

$composantes.append(composante)$

return composantes

2.4 Fonction nettoyage(aretes, N)

Cette fonction est le cœur de l'algorithme 1. Elle nettoie le graphe et le rend connexe.

[H] Nettoyage du graphe **Input** : aretes : liste des arêtes brutes, N : nombre de sommets

Output : Un graphe G propre et connexe

$G \leftarrow$ nouveau graphe vide $G.add_nodes([0, 1, \dots, N - 1])$

Étape 1 : Suppression des boucles et des multi-arêtes **pour** chaque (u, v) dans aretes **faire**

si $u \neq v$ **alors**

$G.add_edge(u, v)$

Étape 2 : Détection des composantes $composantes \leftarrow$ trouver_composantes(G)

Étape 3 : Rétablissement de la connexité **si** $len(composantes) > 1$ **alors**

$noeud_ref \leftarrow$ premier élément de composantes[0] **pour** chaque composante dans composantes[1 :

] **faire**

$noeud_comp \leftarrow$ premier élément de composante $G.add_edge(noeud_ref, noeud_comp)$

return G

2.5 Propriétés du graphe nettoyé

Après exécution de l'algorithme 1, le graphe G' obtenu possède les propriétés suivantes :

- G' est connexe
- G' ne contient aucune boucle
- G' ne contient aucune arête multiple

3 Algorithme 2 : Classification du graphe

3.1 Implémentation Python

```

1 import networkx as nx
2 from algo1_nettoyage import nettoyage
3 from graphe import generate_random_graph
4
5 def colorier_graphe(G):
6
7     sorted_sommet = sorted(
8         G.nodes(),
9         key=lambda x: G.degree(x),

```

```

10         reverse=True
11     )
12
13     couleurs = {}
14
15     for sommet in sorted_sommet:
16
17         couleurs_voisins = {
18             couleurs[voisin]
19             for voisin in G.neighbors(sommet)
20             if voisin in couleurs
21         }
22
23         couleur = 0
24         while couleur in couleurs_voisins:
25             couleur += 1
26
27         couleurs[sommet] = couleur
28
29     return couleurs
30
31 def identifier_sous_graphes(G):
32     visites = set()
33     sous_graphes = []
34     identifiant = 0
35
36     for depart in G.nodes():
37         if depart in visites:
38             continue
39
40         # Parcours en largeur depuis ce sommet
41         composante = set()
42         file = [depart]
43
44         while file:
45             sommet = file.pop(0)
46
47             if sommet in visites:
48                 continue
49
50             visites.add(sommet)
51             composante.add(sommet)
52
53             for voisin in G.neighbors(sommet):
54                 if voisin not in visites:
55                     file.append(voisin)
56
57         # Stocker les infos de cette composante
58         sous_graphes.append({
59             "id": identifiant,
60             "noeuds": sorted(composante),
61             "taille": len(composante),
62             "nb_aretes": G.subgraph(composante).number_of_edges()
63         })
64         identifiant += 1
65
66     return sous_graphes
67
68 def detecter_cliques(G, taille_min=3):
69
70     toutes_cliques = []
71
72     def bron_kerbosch(en_cours, candidats, exclus):
73
74         # Si plus aucun candidat ni exclu alors clique maximale trouvee
75         if not candidats and not exclus:
76             if len(en_cours) >= taille_min:
77                 toutes_cliques.append(sorted(en_cours))
78             return
79
80         for sommet in list(candidats):
81
82             # Voisins de ce sommet

```

```

83     voisins = set(G.neighbors(sommet))
84
85     # On approfondit avec ce sommet dans la clique
86     bron_kerbosch(
87         en_cours | {sommet},      # on ajoute le sommet
88         candidats & voisins,      # candidats = voisins communs
89         exclus & voisins          # exclus = voisins communs
90     )
91
92     # Ce sommet est traite : on le deplace dans exclus
93     candidats = candidats - {sommet}
94     exclus = exclus | {sommet}
95
96     # Lancement initial : clique vide, tous les sommets candidats
97     bron_kerbosch(
98         en_cours = set(),
99         candidats = set(G.nodes()),
100        exclus = set()
101    )
102
103    # Trier les cliques par taille décroissante
104    cliques = sorted(toutes_cliques, key=len, reverse=True)
105
106    # Index inverse : a quelles cliques appartient chaque sommet ?
107    appartenance = {n: [] for n in G.nodes()}
108    for i, c in enumerate(cliques):
109        for n in c:
110            appartenance[n].append(i)
111
112    return cliques, appartenance

```

Listing 2 – algo2_classification.py

3.2 Coloration gloutonne

3.2.1 Fonction colorier_graphe(G)

Cette fonction attribue une couleur à chaque sommet en respectant la contrainte que deux sommets adjacents n'ont pas la même couleur.

[H] Coloration gloutonne par degré décroissant **Input** : G : un graphe propre

Output : Un dictionnaire associant à chaque sommet sa couleur

Tri des sommets par degré décroissant $sommets_tries \leftarrow \text{tri}(G.nodes(), \text{ordre décroissant du degré})$
 $couleurs \leftarrow \{\}$

pour chaque sommet dans $sommets_tries$ **faire**

Collecte des couleurs des voisins déjà colorés $couleurs_voisins \leftarrow \emptyset$

pour chaque voisin dans $G.neighbors(sommet)$ **faire**

si $voisin \in couleurs$ **alors**

$couleurs_voisins.add(couleurs[voisin])$

Recherche de la plus petite couleur disponible $couleur \leftarrow 0$ **tant que** $couleur \in couleurs_voisins$ **faire**

$couleur \leftarrow couleur + 1$

$couleurs[sommet] \leftarrow couleur$

return $couleurs$

3.3 Identification des sous-graphes

3.3.1 Fonction identifier_sous_graphes(G)

Cette fonction identifie les différentes composantes connexes du graphe et en extrait des métriques.

[H] Identification des sous-graphes connexes **Input** : G : un graphe propre

Output : Une liste contenant les informations de chaque sous-graphe

$visites \leftarrow \emptyset$ $sous_graphes \leftarrow []$ $identifiant \leftarrow 0$

```

pour chaque depart dans  $G.nodes()$  faire
  si depart  $\in$  visites alors
    continuer
  composante  $\leftarrow \emptyset$  file  $\leftarrow$  [depart]
  tant que file  $\neq []$  faire
    sommet  $\leftarrow$  file.pop(0)
    si sommet  $\in$  visites alors
      continuer
    visites.add(sommet) composante.add(sommet)
    pour chaque voisin dans  $G.neighbors(sommet)$  faire
      si voisin  $\notin$  visites alors
        file.append(voisin)
    sous_graphes.append({ "id" : identifiant,
                          "noeuds" : tri(composante),
                          "taille" : len(composante),
                          "nb_aretes" : G.subgraph(composante).number_of_edges()
                        }) identifiant  $\leftarrow$  identifiant + 1
return sous_graphes

```

3.4 Détection des cliques par Bron-Kerbosch

3.4.1 Fonction detecter_cliques(G, taille_min)

Cette fonction trouve toutes les cliques maximales de taille supérieure ou égale à un seuil donné.

[H] Bron-Kerbosch : recherche des cliques maximales **Input** : G : graphe, $taille_min$: taille minimale de clique à conserver

Output : *cliques* : liste des cliques trouvées, *appartenance* : dictionnaire

toutes_cliques $\leftarrow []$

Fonction *BronKerbosch*(R, P, X)

// R : clique en construction, P : candidats, X : exclus

si $P = \emptyset$ et $X = \emptyset$ **alors**

si $len(R) \geq taille_min$ **alors**

toutes_cliques.append(tri(R))

return

pour chaque sommet dans une copie de P **faire**

voisins $\leftarrow$ ensemble des voisins de *sommet* *BronKerbosch*($R \cup \{sommet\}, P \cap voisins, X \cap$

voisins) $P \leftarrow P \setminus \{sommet\}$ $X \leftarrow X \cup \{sommet\}$

Lancement initial *BronKerbosch*($\emptyset$, tous les sommets de $G, \emptyset$)

Tri des cliques par taille décroissante *cliques* $\leftarrow$ tri(*toutes_cliques*, ordre décroissant de taille)

Calcul de l'appartenance des sommets aux cliques *appartenance* $\leftarrow \{n : [] \text{ pour } n \text{ dans } G.nodes()\}$

pour chaque i, c dans énumérer(*cliques*) **faire**

pour chaque n dans c **faire**

appartenance[n].append(i)

return (*cliques*, *appartenance*)

3.5 Interprétation criminologique des deux algorithmes

Algorithme	Rôle algorithmique	Interprétation criminologique
Algorithme 1 : Nettoyage	Supprime les anomalies structurelles	Élimine les fausses relations et relie les groupes
Coloration gloutonne	Attribue des classes de sommets	Identifie les niveaux structurels équivalents
Bron-Kerbosch	Détecte les groupes où tous se connaissent	Identifie les cellules opérationnelles soudées

TABLE 1 – Synthèse des deux algorithmes fondamentaux